TP

Les listes

PRÉAMBULE

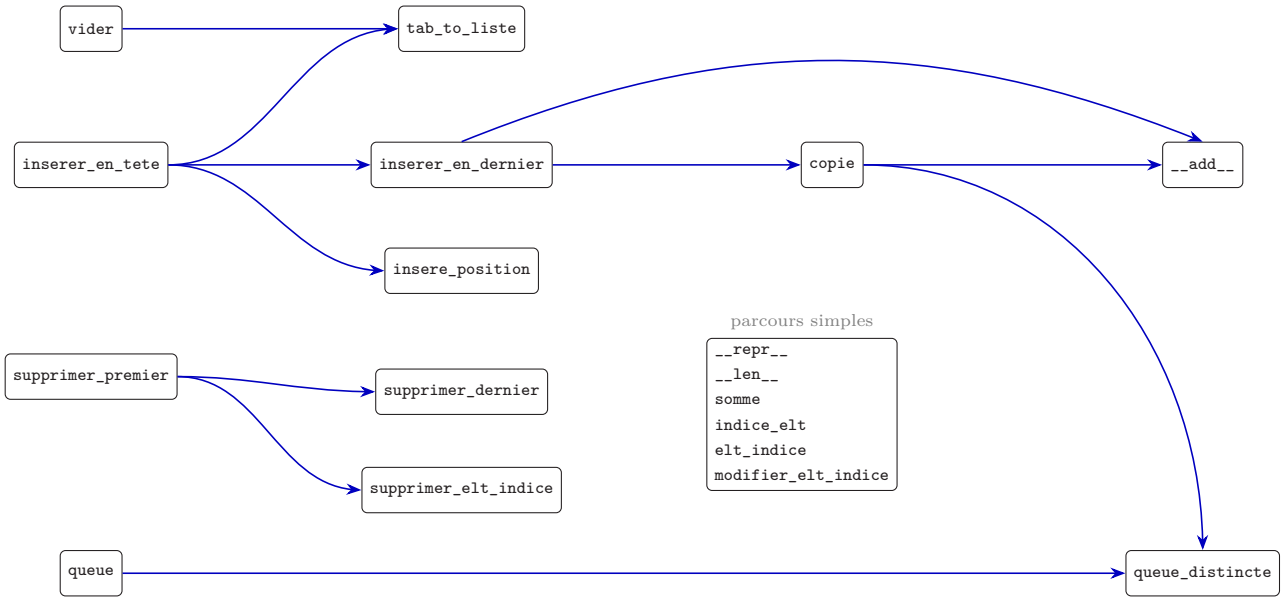
Dans ce TP, on passe de la spécification (le TAD `Liste`) à une implémentation en Python orienté objet, à l'aide de deux classes : `Maillon` et `ListeChainee`. Chaque paragraphe de ce document correspond à un paragraphe du notebook.

1. Du TAD aux classes

Les cinq primitives du TAD `Liste` vues dans le cours vont devenir des méthodes de la classe `ListeChainee` :

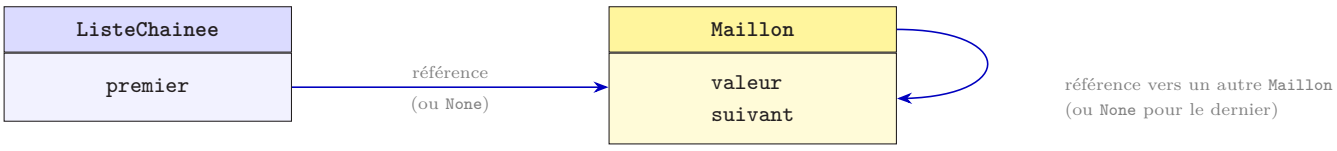
Primitive du TAD	Méthode Python
`Liste()`	`ListeChainee()`
`estVide()`	`L.est_vide()`
`inserer(liste,element)`	`L.inserer_en_tete(elt)`
`tete()`	`L.tete()`
`queue()`	`L.queue()`

Le notebook va plus loin : il ajoute des méthodes pour parcourir, rechercher, supprimer, copier et concaténer. La figure suivante est la feuille de route du TP : une flèche $A \to B$ signifie « la méthode B utilise la méthode A ». Toute méthode qui doit traiter le cas de la liste vide utilise aussi `est_vide`.



2. Anatomie d'une liste chaînée

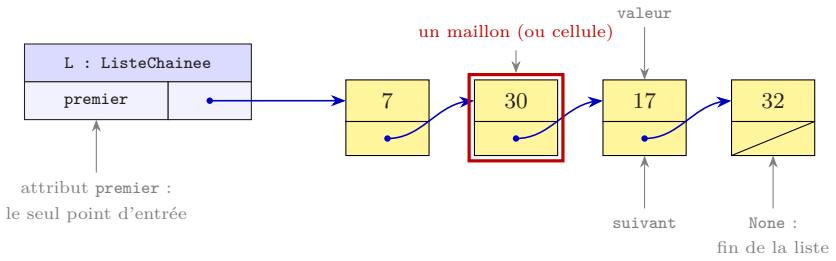
Une liste chaînée met en jeu deux sortes d'objets. Le diagramme ci-dessous précise ce que contient chacun d'eux.



Q1. Dans ce diagramme, quelle flèche traduit la définition récursive d'une liste chaînée donnée dans le cours ?

Q2. La classe `ListeChainee` possède-t-elle un attribut qui contienne des valeurs ? Où sont donc stockées toutes les valeurs de la liste ?

Voici maintenant ce que l'on obtient en mémoire pour la liste $7 \to 30 \to 17 \to 32$:



Q3. Combien y a-t-il d'objets `ListeChainee` dans cette figure ? Combien de `Maillon` ?

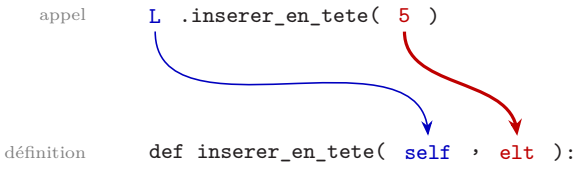
Q4. Quel attribut permet de reconnaître le dernier maillon ? Quelle valeur contient-il ?

À retenir

Une liste chaînée est entièrement déterminée par son premier maillon : si l'on perd la référence vers ce maillon, on perd toute la liste. Le dernier maillon est celui dont `suivant` vaut `None`.

3. Comment fonctionne le notebook ?

Dans le notebook, chaque méthode est d'abord écrite comme une fonction dont le premier paramètre est `self`, puis rattachée à la classe par une affectation du type `ListeChainee.tete = tete`. Quand on écrit `L.inserer_en_tete(5)`, Python transmet l'objet `L` au paramètre `self` :



Indications

Dans le corps d'une méthode, l'objet sur lequel on travaille s'appelle toujours `self`. Le premier maillon de la liste est donc `self.premier`, le maillon suivant le premier est `self.premier.suivant`, etc.

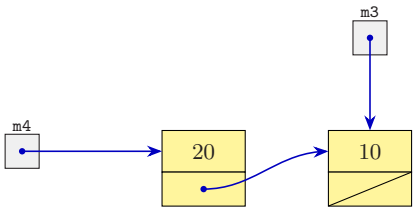
4. Étape 1 : les briques de base

Méthode 1 `Maillon.__init__(self, valeur=None, suivant=None)`

Un maillon est créé avec une valeur et, éventuellement, la référence du maillon qui le suit. Les instructions

```
m3 = Maillon(10)
m4 = Maillon(20, m3)
```

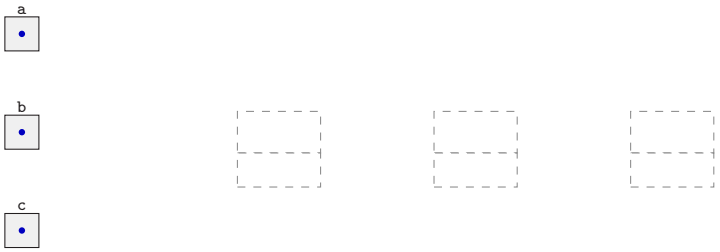
produisent la situation suivante en mémoire (une variable est dessinée comme une petite case avec un point, qui est le départ d'une flèche) :



Q5. Après `m1 = Maillon(5)`, que valent `m1.valeur` et `m1.suivant` ?

Q6. À dessiner. Pour le script ci-dessous, placer les valeurs dans les maillons et dessiner toutes les flèches.

```
a = Maillon(1)
b = Maillon(2, a)
c = Maillon(3, b)
```



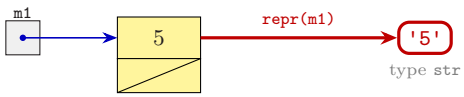
Q7. En partant de `c` et en suivant les liens, dans quel ordre lit-on les valeurs ?

Piège classique

`Maillon(20, 10)` ne provoque aucune erreur, mais l'attribut `suivant` doit contenir un maillon (ou `None`), jamais une valeur. L'erreur n'apparaîtra que plus tard, lors d'un parcours : `'int' object has no attribute 'suivant'`.

Méthode 2 `Maillon.__repr__(self)`

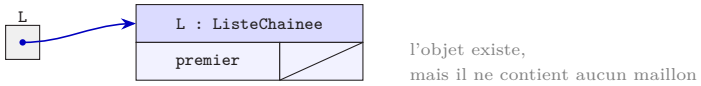
Pour afficher un objet, Python appelle sa méthode `__repr__`, qui doit renvoyer une chaîne de caractères (et non l'afficher avec `print`). Ici, un maillon s'affiche simplement par sa valeur.



Q8. Quel est le type de `repr(m1)` ? Pourquoi `return self.valeur` provoque-t-il une erreur quand la valeur est un entier ?

Méthode 3 `ListeChainee.__init__(self)`

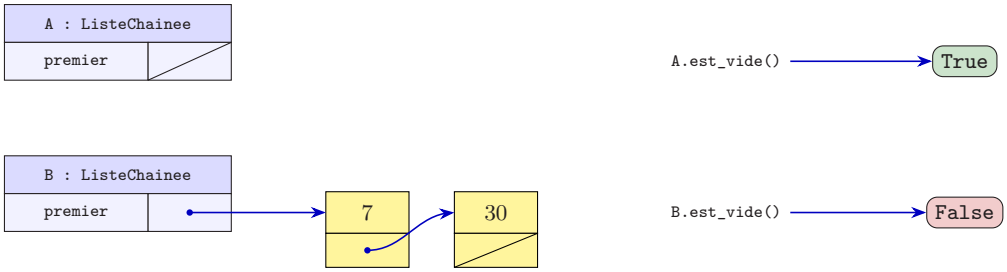
Une liste chaînée vide ne contient aucun maillon : son attribut `premier` vaut `None`. Après `L = ListeChainee()` :



Q9. Que vaut `L.premier` ? La variable `L` vaut-elle `None` ?

5. Étape 2 : les opérations du TAD en $O(1)$

Méthode 4 `est_vide(self)`



Q10. Qu'est-ce qui distingue, dans la figure, une liste vide d'une liste non vide ?

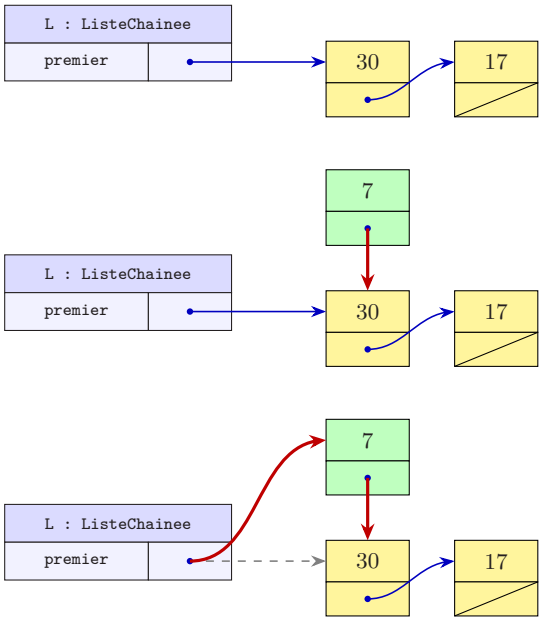
Q11. Faut-il écrire `self.premier == None` ou `self.premier is None` ? Pourquoi est-ce préférable ?

Piège classique

Dans le corps de la méthode, la liste s'appelle `self`, jamais `L`. Écrire `L.premier` fonctionne par hasard tant qu'il existe une variable globale `L`, mais teste alors toujours cette liste-là, quelle que soit celle sur laquelle on appelle la méthode. Les premiers tests passent, puis des erreurs incompréhensibles apparaissent plus loin (par exemple dans `copie`).

Méthode 5 `inserer_en_tete(self, elt)`

Insérer en tête consiste à raccrocher un nouveau maillon devant l'ancien premier. Voici les trois temps de l'opération, pour `L.inserer_en_tete` avec $L = 30 \to 17$:



1 état initial : `L` contient $30 \to 17$.

2 on crée le maillon : `Maillon(elt, self.premier)`. Il pointe vers l'ancien premier.

3 on redirige `self.premier` vers le nouveau maillon.

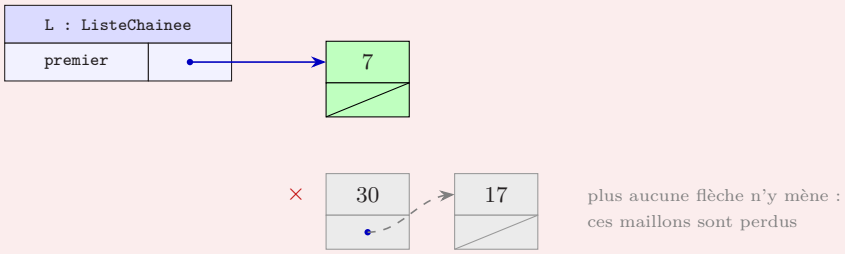
Q12. Dans le temps 2, vers quel maillon pointe l'attribut `suivant` du nouveau maillon ? Quelle expression Python désigne ce maillon ?

Q13. Que se passe-t-il si la liste est vide au départ ? Faut-il écrire un cas particulier ?

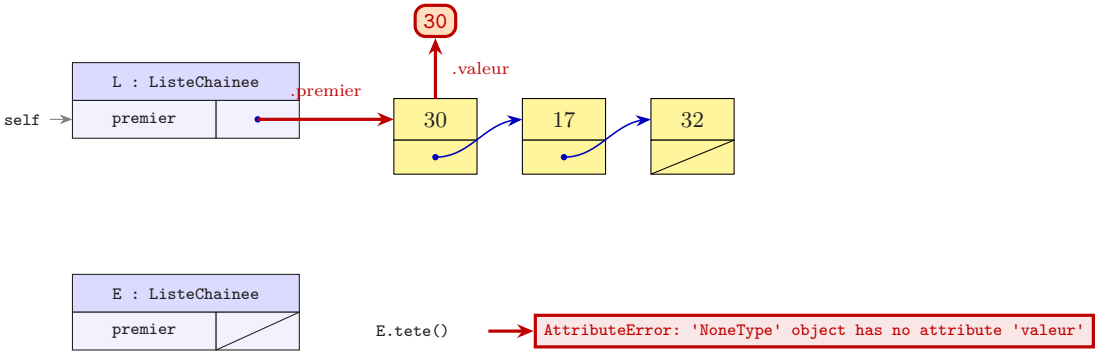
Q14. Redessiner la liste obtenue à la fin, avec ses trois maillons alignés. Le nombre d'opérations dépend-il de la taille de la liste ?

Piège classique

L'ordre des temps 2 et 3 est essentiel. Si l'on modifie `self.premier` avant d'avoir mémorisé l'ancien premier dans le nouveau maillon, plus aucune flèche ne mène à l'ancienne liste : elle est perdue.



Méthode 6 `tete(self)`



Q15. Quel chemin de références suit-on pour atteindre la valeur de tête ?

.....

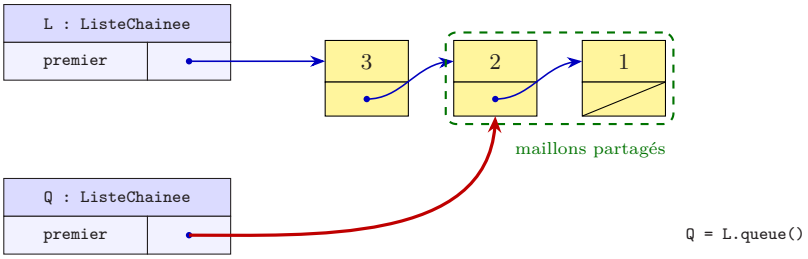
Q16. Que se passe-t-il si l'on appelle `tete()` sur une liste vide ? Lire le message d'erreur et l'expliquer à l'aide du schéma.

.....

.....

Méthode 7 `queue(self)`

La méthode renvoie un nouvel objet `ListeChainee` dont le premier maillon est le deuxième maillon de la liste initiale. Aucun maillon n'est copié : les deux listes se partagent leur fin.

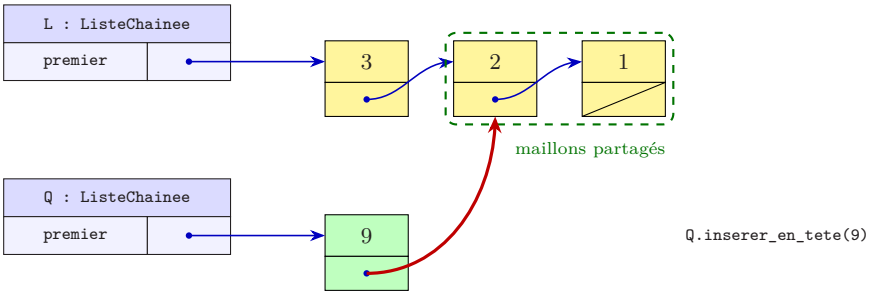


Q17. Après `Q = L.queue()`, combien y a-t-il d'objets `ListeChainee` en mémoire ? Combien de maillons ont été créés par la méthode ?

.....

Q18. On exécute ensuite `Q.inserer_en_tete(9)`. Compléter : $L =$ ..... et $Q =$ ..... Que remarque-t-on sur le schéma ?

.....



Q19. Si l'on modifiait la valeur du maillon 2 en passant par `Q`, la liste `L` serait-elle affectée ? Pourquoi ?

Q20. Que doit renvoyer `queue()` pour une liste d'un seul élément ? Le schéma est-il différent ?

Indications

Créer un nouvel objet `ListeChainee` (vide), puis lui donner pour `premier` le `suivant` du premier maillon. Aucun parcours, aucun appel à `Maillon` : le coût est constant, $O(1)$.

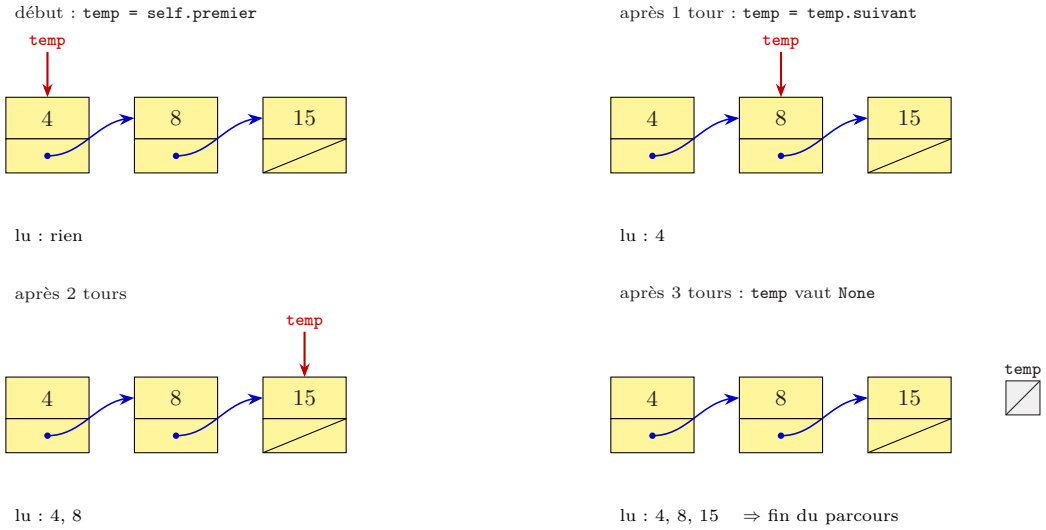
À retenir

Les cinq primitives du TAD `Liste` sont maintenant implémentées et toutes en $O(1)$. Le reste du TP consiste à parcourir la chaîne maillon par maillon.

6. Étape 3 : apprendre à parcourir une liste

Méthode 8 `ListeChainee.__repr__(self)`

Pour afficher la liste sous la forme `[4, 8, 15]`, il faut passer par tous les maillons. Comme on ne peut pas « sauter » directement au maillon voulu, on utilise une variable auxiliaire `temp` qui se déplace de maillon en maillon : c'est le parcours, le schéma de base de presque toutes les méthodes qui suivent.



Le squelette d'un parcours est toujours le même :

```
temp = self.premier
while ... :
    ...           # on utilise temp.valeur
    temp = ...    # on avance d'un maillon
```

Q21. Quelle condition faut-il écrire dans le `while` pour passer par tous les maillons ? Que vaut `temp` quand la boucle se termine ?

Q22. Quelle instruction fait avancer `temp` d'un maillon ? Combien de tours de boucle pour une liste de n maillons ? En déduire le coût du parcours.

Q23. Pour obtenir `[4, 8, 15]`, il faut une virgule après chaque valeur sauf la dernière. Comment reconnaître, pendant le parcours, que `temp` désigne le dernier maillon ?

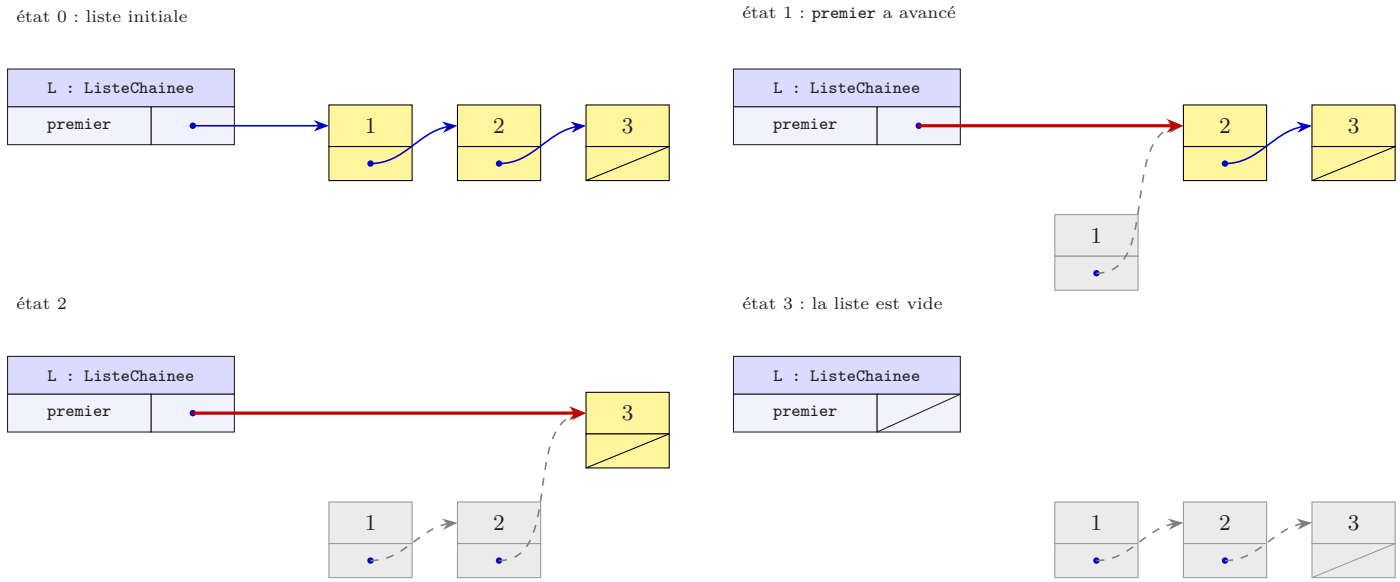
.....

Indications

Construire une chaîne de caractères en partant de `"["`, y ajouter `str(temp.valeur)` à chaque tour (suivie de `", "` si ce n'est pas le dernier maillon), puis terminer par `"]"`. La méthode doit renvoyer la chaîne.

Méthode 9 `vider(self)`

Vider la liste, c'est retirer ses maillons un par un, en partant de la tête. À chaque étape, `premier` avance d'un maillon et le maillon laissé derrière n'est plus atteint par aucune flèche (il est en gris sur la figure).



Q24. Combien de fois doit-on répéter l'opération « faire avancer `premier` » pour une liste de n maillons ? Quelle méthode déjà écrite permet de savoir quand s'arrêter ?

.....

Q25. À la fin, que contient `premier` ? Les trois maillons existent-ils encore, et qui peut encore les atteindre ?

.....

.....

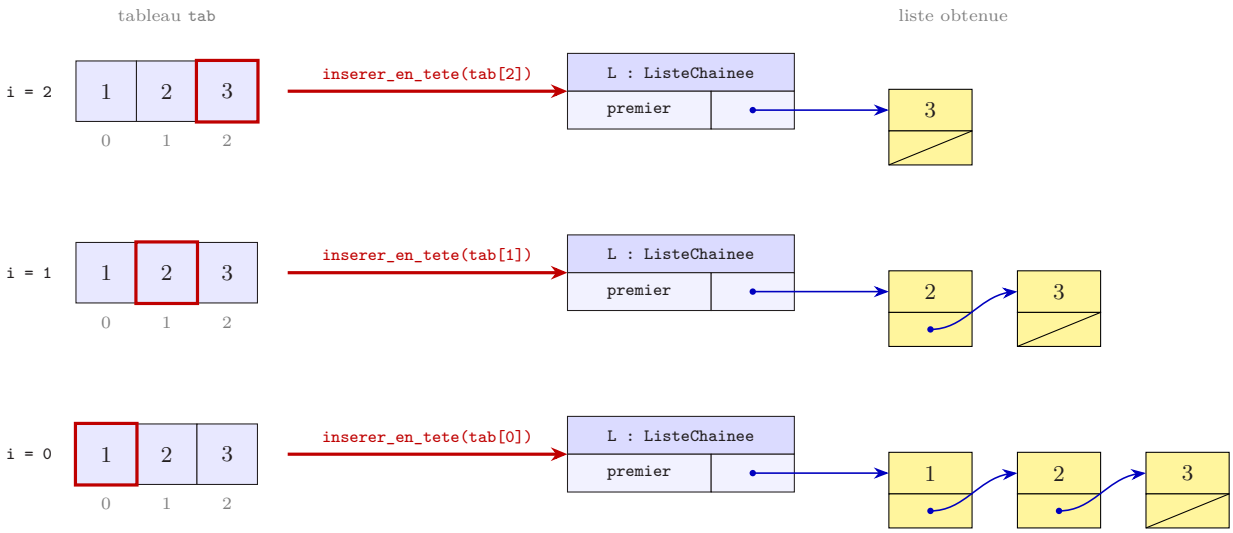
Q26. Pour aller plus loin. Une seule instruction `self.premier = None` suffirait-elle à vider la liste ? Que se passerait-il pour une liste `Q = L.queue()` créée auparavant ?

.....

.....

Méthode 10 `tab_to_liste(self, tab)`

On veut transformer un tableau `tab` en liste, dans le même ordre. La seule opération d'insertion dont on dispose est `inserer_en_tete` : chaque nouvel élément se place devant les précédents.



Q27. Dans quel ordre faut-il insérer en tête les éléments de `tab = [1, 2, 3]` pour obtenir la liste $1 \to 2 \to 3$?

Q28. Quelle liste obtiendrait-on en parcourant le tableau de gauche à droite avec `inserer_en_tete` ?

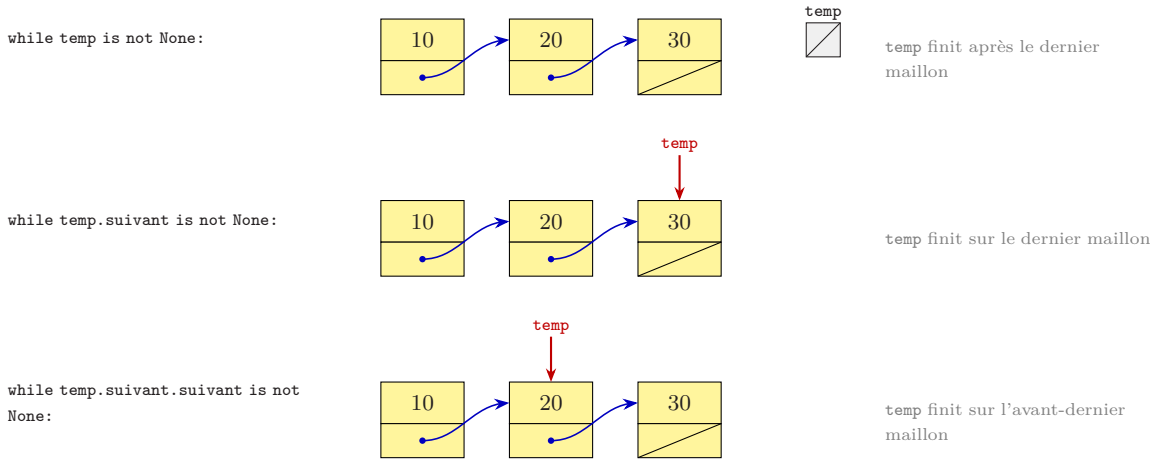
Q29. Comment parcourir les indices de `len(tab) - 1` jusqu'à `0` avec `range` ? On rappelle que `range(a, b, p)` part de `a`, avance du pas `p` et s'arrête avant `b`.

Q30. Pourquoi commencer par `self.vider()` ? Quel est le coût de la méthode ?

7. Étape 4 : agir à la fin de la liste

A. Fiche : où s'arrête `temp` ?

Selon la condition du `while`, la variable `temp` ne s'arrête pas au même endroit. Choisir la bonne condition est la clé des trois méthodes qui suivent.



Q31. Dans quelle situation peut-on accrocher un nouveau maillon à la fin de la liste ? Pourquoi ?

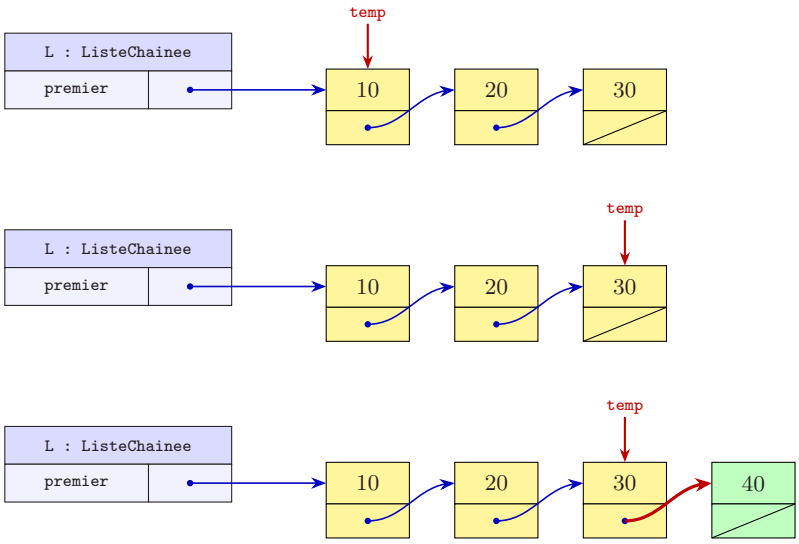
Q32. Que se passe-t-il avec `temp.suivant.suivant` si la liste ne contient qu'un seul maillon ? et avec `temp.suivant` si la liste est vide ?

. . . . . .

. . . . . .

Méthode 11 `inserer_en_dernier(self, elt)`

Insérer à la fin demande de rejoindre le dernier maillon pour lui accrocher le nouveau.



1 `temp = self.premier`

2 la boucle s'arrête : `temp.suivant is None`

3 `temp.suivant = Maillon(elt)`

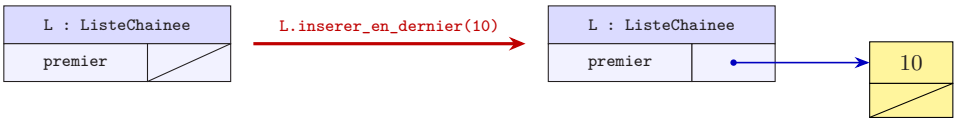
Q33. Quelle valeur a `temp.suivant` lorsque `temp` désigne le dernier maillon ? En déduire la condition d'arrêt du `while`.

. . . . . .

Q34. Le coût de cette méthode est-il le même que celui de `inserer_en_tete` ? Justifier.

. . . . . .

Il reste le cas où la liste est vide :

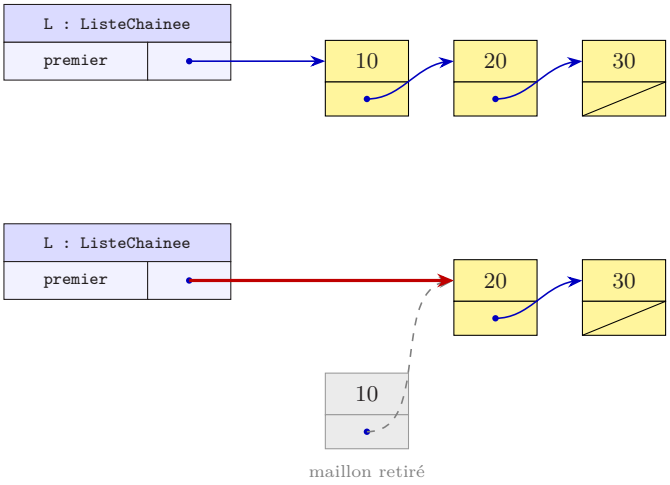


Q35. Pourquoi la boucle de parcours ne peut-elle pas s'exécuter sur une liste vide ? Quelle méthode déjà écrite permet de traiter ce cas sans rien recopier ?

. . . . . .

. . . . . .

Méthode 12 `supprimer_premier(self)`



avant

après `premier` désigne l'ancien deuxième maillon

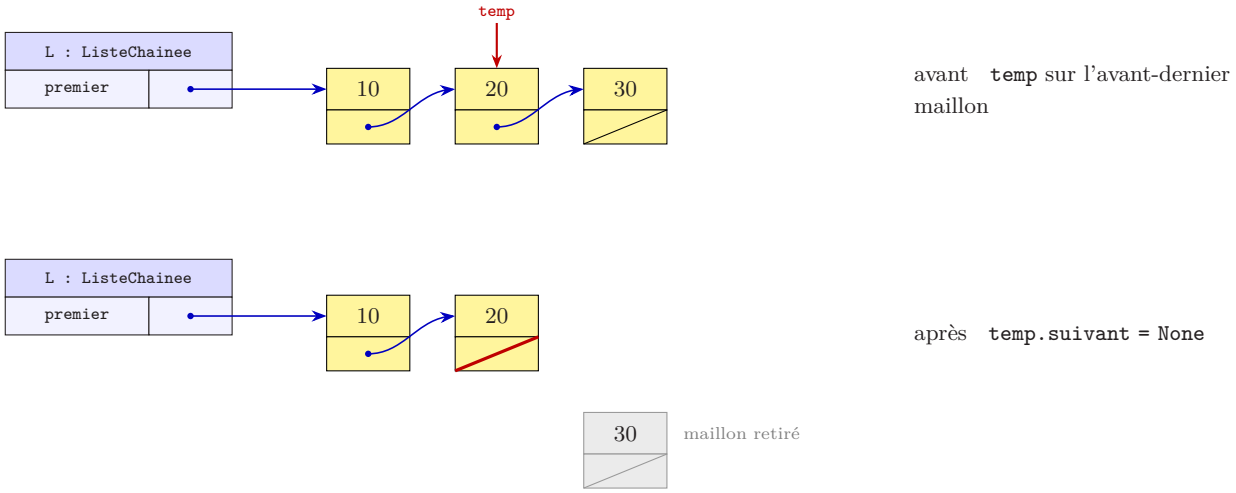
Q36. Quelle instruction fait « avancer » `premier` d'un maillon ?

Q37. Que devient le maillon retiré ? Faut-il le « détruire » explicitement ?

Q38. Que faut-il tester avant de supprimer, et pourquoi ? Quelle est la valeur renvoyée par la méthode ?

Méthode 13 `supprimer_dernier(self)`

Pour retirer le dernier maillon, il faut modifier le `suivant` de l'avant-dernier : on ne peut pas « reculer » dans une liste simplement chaînée, donc `temp` doit s'arrêter avant le maillon à retirer.



Q39. Sur quel maillon la variable `temp` doit-elle s'arrêter ? Quelle condition de boucle y conduit (voir la fiche) ?

Q40. Quels sont les deux cas particuliers à traiter avant la boucle ? Comment le second se ramène-t-il à une méthode déjà écrite ?

Q41. Pourquoi supprimer en queue coûte-t-il $O(n)$ alors que supprimer en tête coûte $O(1)$?

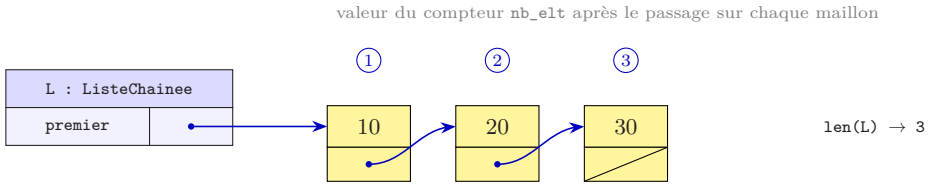
À retenir

Dans une liste simplement chaînée, agir en tête est gratuit, agir en queue est coûteux : il faut d'abord parcourir la chaîne en entier.

8. Étape 5 : accéder aux éléments par leur indice

Méthode 14 `__len__(self)`

Compter les éléments est un parcours : on incrémente un compteur à chaque maillon rencontré.

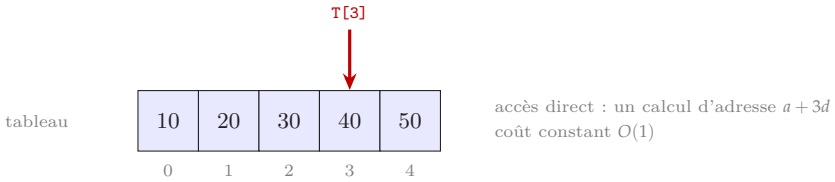


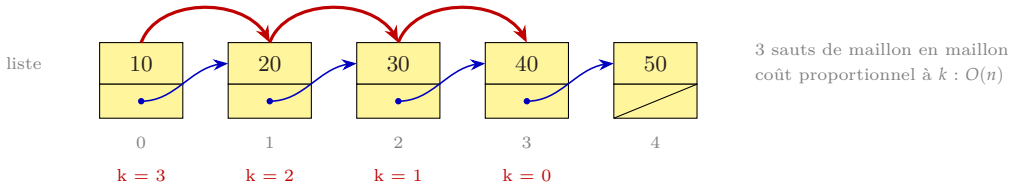
Q42. Quelle est la valeur initiale du compteur ? Que renvoie la méthode pour une liste vide, sans cas particulier ?

Q43. Le coût est $O(n)$. Que faudrait-il ajouter à la classe pour obtenir `len` en $O(1)$, et quel prix paie-t-on ?

Méthode 15 `elt_indice(self, k)`

Dans un tableau, `T[k]` s'obtient en un seul calcul d'adresse $(a + kd)$. Dans une liste chaînée, il faut suivre les liens un par un :





Q44. Combien de « sauts » faut-il pour atteindre l'élément d'indice `k` ? Quel indice est le plus rapide à atteindre, lequel est le plus lent ?

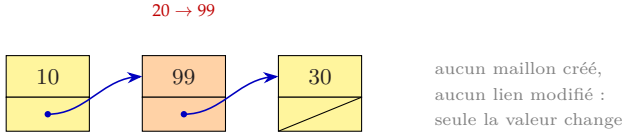
Q45. Quelles sont les deux raisons possibles d'arrêter la boucle ? Que signifie chacune ?

Indications

Décrémenter `k` à chaque saut : `k` est une variable locale, on peut la modifier. À la fin, vérifier avec `assert k == 0` que l'indice était valide.

Méthode 16 `modifier_elt_indice(self, k, elt)`

Même parcours que précédemment, mais on écrit dans le maillon au lieu de le lire. Ici, on remplace par 99 l'élément d'indice 1 :



Q46. Cette méthode crée-t-elle un maillon ? Modifie-t-elle un lien ?

Q47. Quelle partie du code de `elt_indice` peut-on réutiliser telle quelle ? Que change-t-on à la fin ?

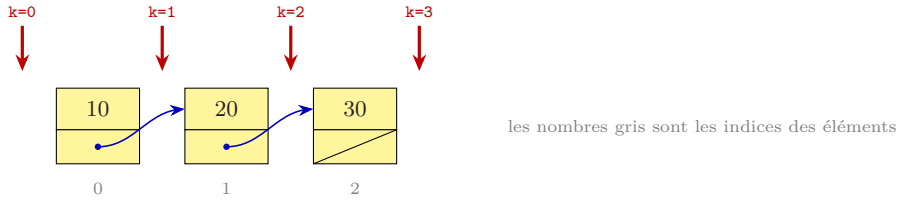
. . . . . .

. . . . . .

9. Étape 6 : insérer, rechercher, supprimer au milieu

Méthode 17 `insere_position(self, k, elt)`

Insérer à l'indice k, c'est placer le nouvel élément avant l'élément qui porte actuellement l'indice k. Pour une liste de trois éléments, voici les positions possibles :

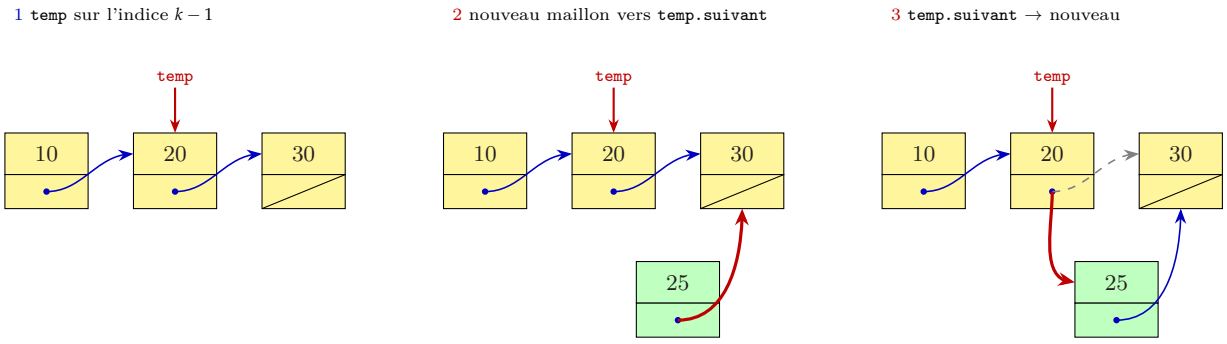


Q48. Combien y a-t-il de positions d'insertion pour une liste de n éléments ? Pour quelles valeurs de k l'insertion est-elle autorisée ?

. . . . . .

. . . . . .

Pour $k \geq 1$, il faut s'arrêter sur le maillon d'indice $k-1$, puis procéder en trois temps (ici $k = 2$, $elt = 25$) :



Q49. Au temps 2, quelle expression Python désigne le maillon vers lequel pointe le nouveau maillon ? Combien de sauts fait `temp` depuis le premier maillon ?

. . . . . .

. . . . . .

Q50. Pourquoi le cas $k = 0$ est-il à part ? Quelle méthode déjà écrite le traite ?

. . . . . .

. . . . . .

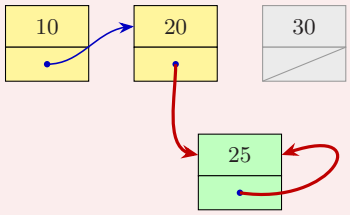
Q51. Que se passe-t-il pour $k =$ `len(L)` ? Faut-il un traitement particulier ?

. . . . . .

. . . . . .

Piège classique

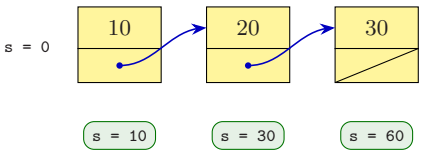
Si l'on exécute le temps 3 avant le temps 2, `temp.suivant` désigne déjà le nouveau maillon : celui-ci pointe alors vers lui-même. Un parcours de la liste ne s'arrête plus jamais, et le maillon 30 est perdu.



le maillon 25 pointe vers lui-même,
le maillon 30 n'est plus atteint

Méthode 18 `somme(self)`

C'est encore un parcours : on cumule les valeurs dans une variable `s`.

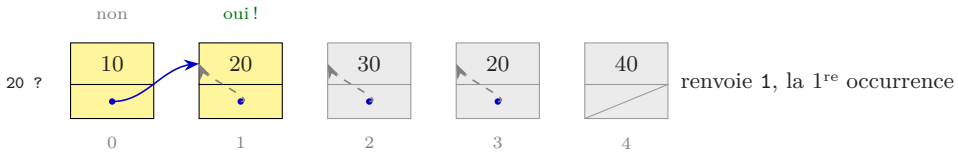


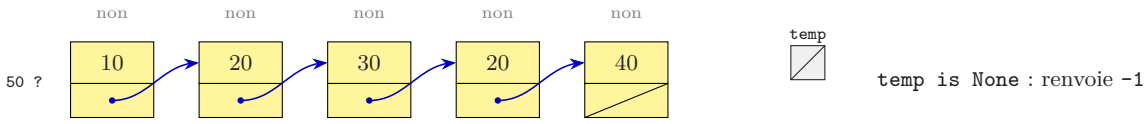
Q52. Pourquoi la valeur initiale de `s` est-elle 0 ? Que renvoie `somme` pour une liste vide ?

Q53. Quelle différence y a-t-il entre le parcours de `somme` et celui de `__repr__` (méthode 8) ?

Méthode 19 `indice_elt(self, elt)`

On parcourt la liste en comptant les maillons, et on s'arrête dès que l'on trouve la valeur cherchée. Recherche de 20, puis de 50 dans la liste $10 \to 20 \to 30 \to 20 \to 40$:



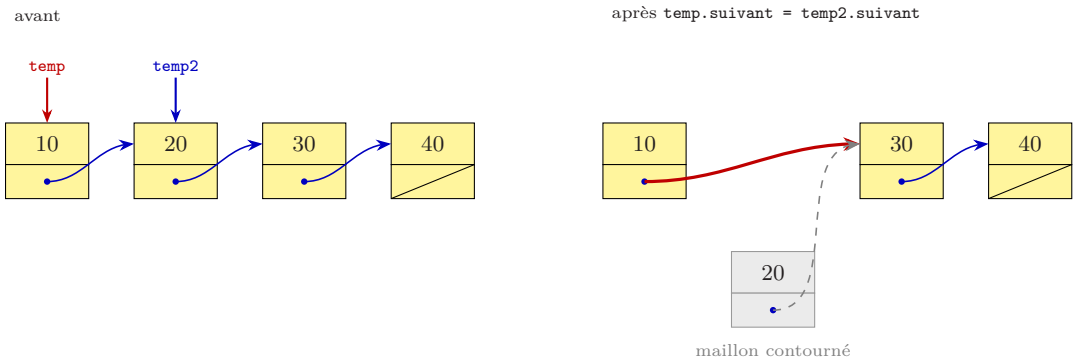


Q54. Que renvoie la méthode si la valeur apparaît plusieurs fois ? Pourquoi peut-on écrire un `return` à l'intérieur de la boucle ?

Q55. Pourquoi choisit-on -1 pour signaler l'absence ? Quels sont les coûts dans le meilleur et dans le pire cas ?

Méthode 20 `supprimer_elt_indice(self, k)`

Pour supprimer le maillon d'indice k, on le contourne : le maillon d'indice $k-1$ pointe désormais vers le maillon d'indice $k+1$. Ici, $k = 1$:



Q56. Sur quel maillon doit s'arrêter `temp` ? Quelle instruction réalise le contournement ?

. . . . . .

. . . . . .

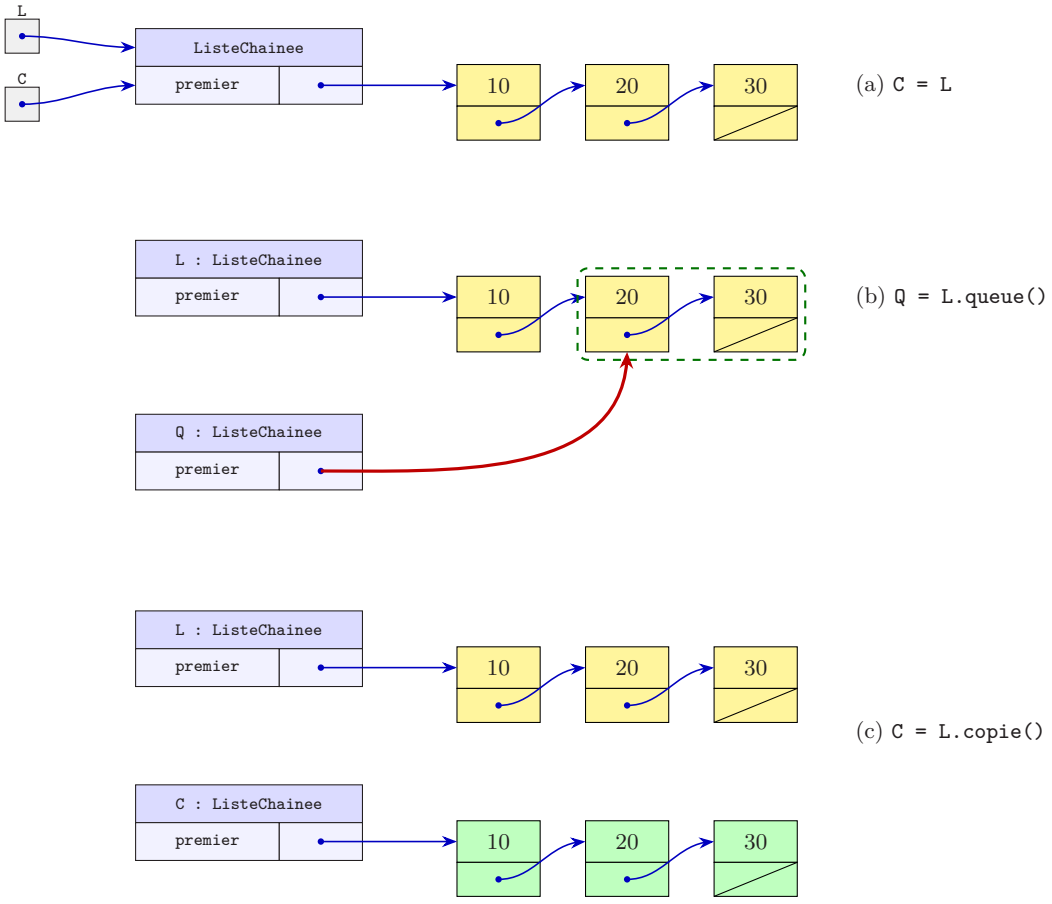
Q57. Pourquoi $k = 0$ est-il un cas à part ? Quel parallèle peut-on faire avec `insere_position` ?

. . . . . .

. . . . . .

10. Étape 7 : copier, concaténer, et comprendre le partage

Trois façons d'obtenir « deux listes » à partir de `L` :



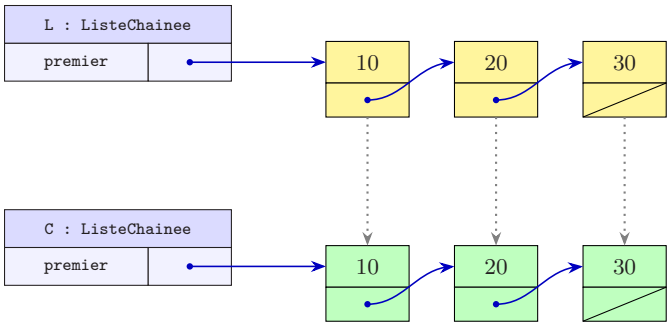
Q58. À compléter. Pour chaque situation, indiquer le résultat de chaque test, puis dire si `L` est modifiée par `C.inserer_en_tete(5)`.

	`C is L`	`C.premier is L.premier`	`C.premier is L.premier.suivant`	`L` modifiée ?
(a) `C = L`				
(b) `Q = L.queue()`				
(c) `C = L.copie()`				

Dans les lignes (b) et (c), *C* désigne l'autre liste (*Q* en (b)).

Méthode 21 `copie(self)`

Une copie indépendante est une liste dont tous les maillons sont neufs (en vert) : seules les valeurs sont recopiées.

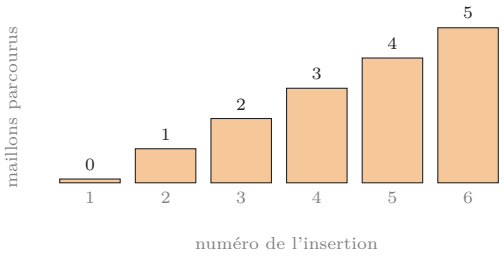


valeurs recopiées, maillons neufs

Q59. Quelle méthode déjà écrite permet de construire la copie maillon par maillon, dans le bon ordre ? Pourquoi `inserer_en_tete` donnerait-elle une liste inversée ?

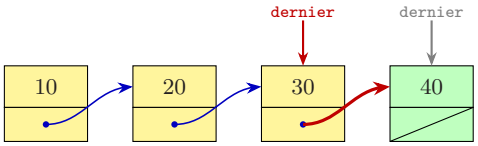
Q60. Cas particulier : que doit renvoyer `copie` pour une liste vide ?

Q61. Pour aller plus loin : le coût. Si `copie` appelle `inserer_en_dernier` à chaque élément, combien de maillons sont parcourus au total pour copier 6 éléments ? Et pour n éléments ?



total $= 0+1+2+3+4+5 = 15$
en général $\dfrac{n(n-1)}{2}$: coût quadratique

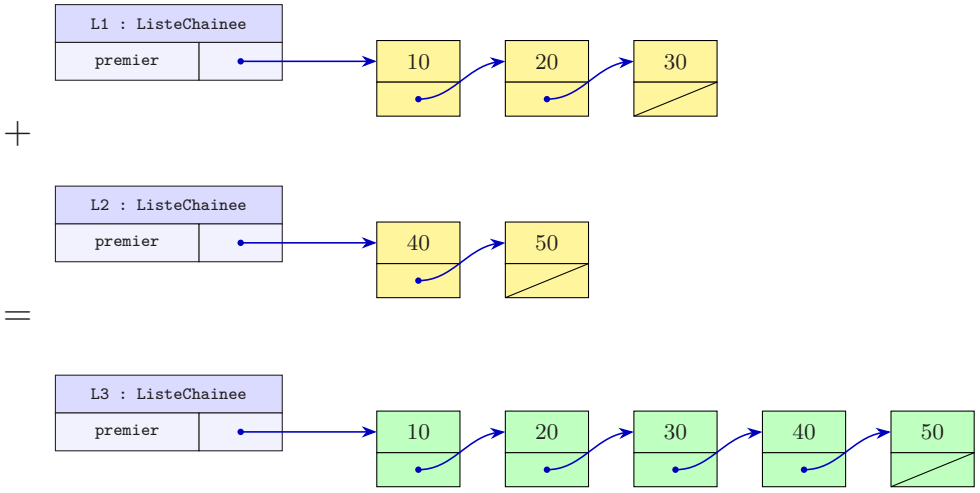
Q62. Comment obtenir une copie en $O(n)$? Le schéma suivant donne une piste.



on garde une référence vers le dernier maillon construit, et on la fait avancer

Méthode 22 `__add__(self, autreListe)`

L'opérateur `+` doit renvoyer une nouvelle liste, sans modifier les deux listes initiales :



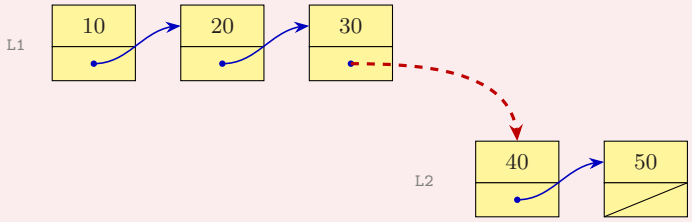
Q63. Comment construire `L3` à l'aide des méthodes `copie` et `inserer_en_dernier` ?

. . . . . .

. . . . . .

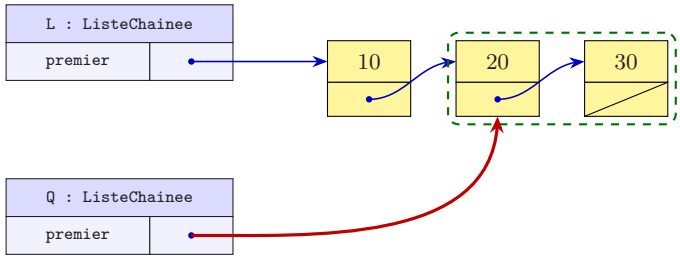
Piège classique

Il est tentant de relier directement le dernier maillon de `L1` au premier de `L2`. Ce serait interdit : `L1` serait modifiée (elle contiendrait 5 éléments) et les deux listes partageraient des maillons.

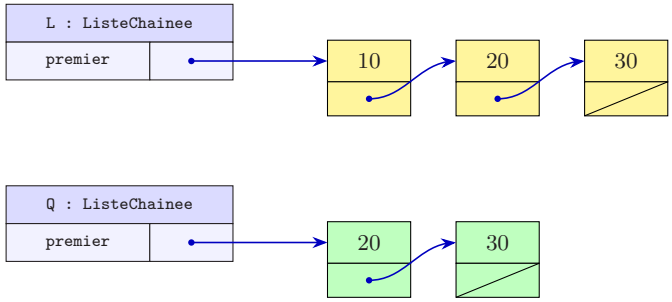


Méthode 23 `queue_distincte(self)`

Contrairement à `queue()`, la liste renvoyée ne partage aucun maillon avec la liste initiale :



`queue()` maillons partagés



`queue_distincte()` maillons neufs

Q64. Écrire `queue_distincte` en une seule ligne à l'aide de deux méthodes déjà écrites.

. . . . . .

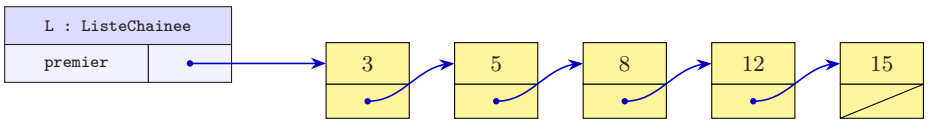
Q65. Comparer `queue()` et `queue_distincte()` : maillons créés, coût, risque d'effet de bord.

11. Bilan

A. Suivre la cellule finale du notebook

La dernière cellule exécute la suite d'instructions ci-dessous. Dessiner l'état de la liste après chacune des deux modifications.

```
L.tab_to_liste([3, 5, 8, 12, 15])
```




```
L.insere_position(2, 7)
```




```
L.supprimer_elt_indice(1)
```

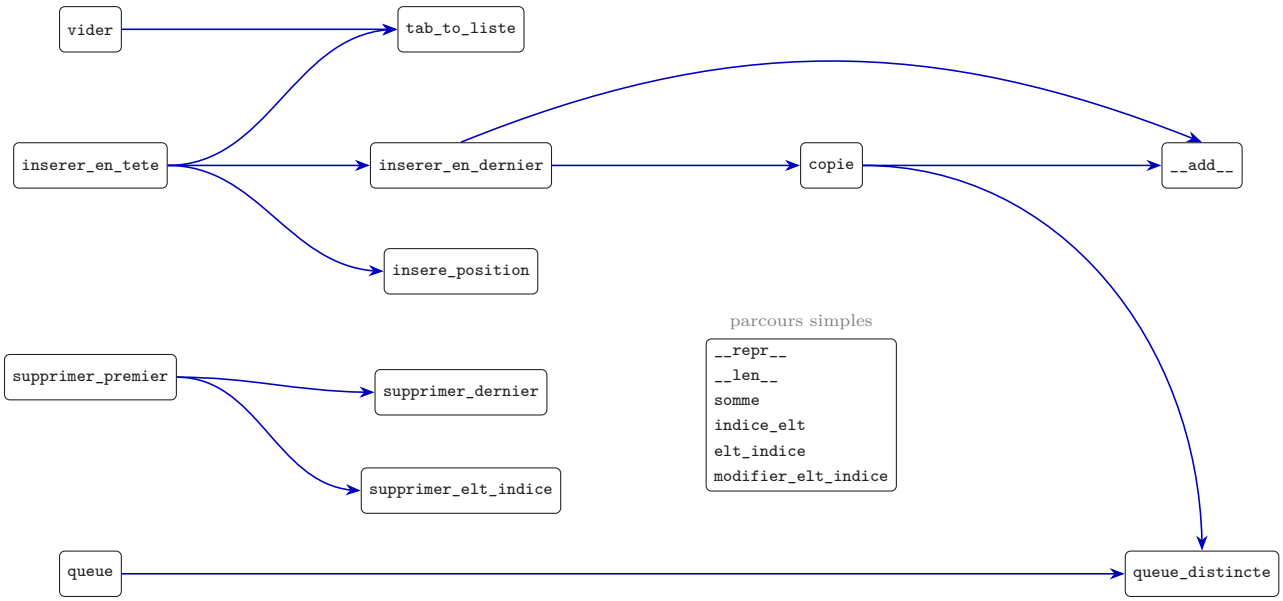


Q66. À compléter. Écrire l'affichage produit par la dernière cellule du notebook.

`print("Liste initiale :", L)`	
`print("Taille :", len(L))`	
`print("Premier élément :", L.tete())`	
`print("Somme :", L.somme())`	
`print("Après insertion :", L)`	
`print("Après suppression :", L)`	
`print("Copie :", C)`	
`print("Concaténation :", F)`	

B. Le coût de chaque méthode

Q67. À colorier. Sur la feuille de route, colorier en vert les méthodes en $O(1)$, en orange celles en $O(n)$ et en rouge celles en $O(n^2)$ telles qu'elles ont été écrites dans le notebook.



Q68. Quelles méthodes pourraient devenir plus efficaces avec une amélioration de la classe (attribut `taille`, référence `dernier`) ?

. . . . . .

. . . . . .

C. Tableau ou liste chaînée ?

Q69. À compléter avec $O(1)$, $O(n)$ ou « impossible » (structure de taille fixe).

	`numpy.array`	`list` Python	liste chaînée
accès à l'indice k			
insertion en tête			
insertion en queue			
suppression en tête			

À retenir

Il n'existe pas de « meilleure » structure : le tableau est imbattable pour l'accès par indice, la liste chaînée pour les ajouts et retraits en tête. Le choix de l'implémentation dépend des opérations que l'on utilisera le plus.